

HW8 report

楊文德

程式說明

在程式一開始我先定義角色可能的走向(直向—LUKA、JOJO，斜向—FIZZ)而定義兩個 `vector<pair<int,int>>` 型態的陣列，用來之後對每個點搜索用。

```
vector<pair<int, int>> incline{{-1, 1}, {1, 1}, {-1, -1}, {1, -1}};  
vector<pair<int, int>> straight{{1, 0}, {-1, 0}, {0, -1}, {0, 1}}; // 四個方向
```

隨後依據各角色間能力的不同我將函數分成三大類:

1. LUKA (DFS_LUKA、DFS_LUKA_Helper)

```
// LUKA 的 DFS 函數入口  
int DFS_LUKA(const vector<vector<int>> &Graph,  
             const pair<int, int> &start,  
             const pair<int, int> &end)  
{  
    int m = Graph.size();  
    int n = Graph[0].size();  
  
    // 訪問矩陣  
    vector<vector<bool>> visited(m, vector<bool>(n, false));  
  
    // 標記起點為已訪問  
    visited[start.first][start.second] = true;  
  
    int path_count = 0;  
  
    // 開始DFS搜索，初始生命值100，無前方向(-1)  
    DFS_LUKA_Helper(Graph, visited, start.first, start.second,  
                    end, 100, -1, path_count);  
  
    return min(path_count, 1000000);  
}
```

```
// LUKA 的 DFS 函數 - 遞歸實現  
int DFS_LUKA_Helper(const vector<vector<int>> &Graph,  
                    vector<vector<bool>> &visited,  
                    int x, int y,  
                    const pair<int, int> &end,  
                    int health,  
                    int last_dir,  
                    int &path_count)
```

首先我定義 `DFS_LUKA` 函數用來計算方法數，用 `visited` 布林二維陣列紀錄地圖是否有被探索過，並用 `DFS_LUKA_Helper` 協助計算方法數，最後輸出時再次確保不超過 1000000。變數上，`DFS_LUKA_Helper` 除了沿用 `DFS_LUKA` 的變數外，還多了 `x,y`(起點)、`health`(生命)、`last_dir`(紀錄上次走的方向，依照題目防止重複)、`path_count`(紀錄可行方案)。

```

// 如果路徑數已達上限，提前終止
if (path_count >= 1000000)
{
    return path_count;
}

// 到達終點且生命值 > 0
if (x == end.first && y == end.second && health > 0)
{
    path_count++;
    return path_count;
}

```

在 Helper 函數中我先設定方法數的上限為 1000000 作為邊界，防止程式跑太久導致浪費時間，並在之後檢測是否到終點並確認角色是否活著，確認後就增加方法數並回傳。

```

// 檢查四個方向
for (int dir = 0; dir < 4; dir++)
{
    // LUKA 不能連續朝同一方向移動
    if (dir == last_dir)
        continue;

    int nx = x + straight[dir].first;
    int ny = y + straight[dir].second;

    // 檢查邊界
    if (nx < 0 || nx >= Graph.size() || ny < 0 || ny >= Graph[0].size())
    {
        continue;
    }

    // 檢查是否為樹木或已訪問
    if (Graph[nx][ny] == 0 || visited[nx][ny])
    {
        continue;
    }

    // 計算移動後的生命值
    int new_health = health - 2; // 基本移動消耗
    if (Graph[nx][ny] == 2)
    { // 踩到毒菇額外扣除10點
        new_health -= 10;
    }

    // 如果不是終點但生命值已不足，跳過
    if (new_health <= 0)
    {
        continue;
    }

    // 標記訪問並遞歸搜索
    visited[nx][ny] = true;
    DFS_LUKA_Helper(Graph, visited, nx, ny, end, new_health, dir, path_count);
    visited[nx][ny] = false; // 回溯
}

return path_count;

```

確認沒有要回傳/終止的話就在原有的點上加上四個方向，之後依序確認是否超出邊界、是否為樹木或以探索過，樹林路徑確認好之後就開始計算損失的血量，如果角色會死亡就結束往這方向的探索，如過依然還活著就更新遞迴函數至新的點並累加。

- **注意！** 因為這題目變形至要計算可行的路徑數，而非平常的那種單純找路徑而已，所以搜索過後還是要回朔，過程如下。

```
// 標記訪問並遞歸搜索
visited[nx][ny] = true;
DFS_LUKA_Helper(Graph, visited, nx, ny, end, new_health, dir, path_count);
visited[nx][ny] = false; // 回溯
```

更新點時要視原本的點探索過了，但是為了下一輪的計算，必須在這一輪結束後回溯至尚未探索的狀態。

2. FIZZ (DFS_FIZZ、DFS_FIZZ_Helper)

```
int DFS_FIZZ(const vector<vector<int>> &Graph,
             const pair<int, int> &start,
             const pair<int, int> &end)
{
    int m = Graph.size();
    int n = Graph[0].size();

    // 訪問矩陣
    vector<vector<bool>> visited(m, vector<bool>(n, false));

    // 標記起點為已訪問
    visited[start.first][start.second] = true;

    int path_count = 0;

    // 開始DFS搜索，初始生命值100，無前方向(-1)
    DFS_FIZZ_Helper(Graph, visited, start.first, start.second,
                    end, 50, path_count);

    return min(path_count, 1000000);
}
```

```
// FIZZ 的 DFS 搜尋實作
int DFS_FIZZ_Helper(const vector<vector<int>> &Graph,
                    vector<vector<bool>> &visited,
                    int x, int y,
                    const pair<int, int> &end,
                    int health,
                    int &path_count)
```

在輸出函數上與 LUKA 一致，在 Helper 上因為 FIZZ 不需考慮與原本行徑方向等條件，所以相對上少了 last_dir 紀錄先前的方向。但是在 Helper 函數實做上卻有很大的差別：

```

// 先檢查是否有可以訪問的毒菇
bool found_mushroom = false;

// 第一階段：檢查和訪問毒菇
for (int dir = 0; dir < 4; dir++)
{
    int nx = x + incline[dir].first;
    int ny = y + incline[dir].second;

    // 檢查邊界
    if (nx < 0 || nx >= Graph.size() || ny < 0 || ny >= Graph[0].size())
        continue;

    // 檢查是否為未訪問的毒菇
    if (Graph[nx][ny] == 2 && !visited[nx][ny])
    {
        found_mushroom = true;
        int new_health = health - 11; // 斜向移動(1) + 毒菇懲罰(10)

        if (new_health > 0)
        {
            visited[nx][ny] = true;
            DFS_FIZZ_Helper(Graph, visited, nx, ny, end, new_health, path_count);
            visited[nx][ny] = false;
        }
    }
}

// 第二階段：如果沒有找到可訪問的毒菇，嘗試普通移動
if (!found_mushroom)
{
    // 進行普通移動
    for (int dir = 0; dir < 4; dir++)
    {
        int nx = x + incline[dir].first;
        int ny = y + incline[dir].second;

        // 檢查邊界和可行性
        if (nx < 0 || nx >= Graph.size() || ny < 0 || ny >= Graph[0].size())
            continue;

        // 檢查是否為可行路徑且未訪問
        if (Graph[nx][ny] >= 1 && !visited[nx][ny])
        {
            int new_health = health - 1; // 斜向移動消耗1點生命值

            if (new_health > 0)
            {
                visited[nx][ny] = true;
                DFS_FIZZ_Helper(Graph, visited, nx, ny, end, new_health, path_count);
                visited[nx][ny] = false;
            }
        }
    }
}
}

```

根據 FIZZ 會優先探索附近的毒蘑菇並優先探索、斜向走路等等規則，我用布林變數將函數區塊分成兩部分，如果在斜向的位置上找到讀蘑菇就只走毒蘑菇，反之亦然，之後的邏輯與 LUKA 的判斷一致。

3. JOJO (DFS_JOJO、DFS_JOJO_Helper)

```
int DFS_JOJO(const vector<vector<int>> &Graph,
             const pair<int, int> &start,
             const pair<int, int> &end)
{
    vector<vector<bool>> visited(Graph.size(), vector<bool>(Graph[0].size(), false));
    visited[start.first][start.second] = true;
    int path_count = 0;

    // 開始DFS搜索，初始生命值100，非忍者模式，忍者次數0
    DFS_JOJO_Helper(Graph, visited, start.first, start.second, end, 100, false, 0, path_count);

    return min(path_count, 1000000);
}
```

```
// JOJO 的 DFS 搜尋實作
int DFS_JOJO_Helper(const vector<vector<int>> &Graph,
                   vector<vector<bool>> &visited,
                   int x, int y,
                   const pair<int, int> &end,
                   int health,
                   int stand_turns,
                   bool stand_used,
                   int &path_count)
```

相較於之前的 LUKA、FIZZ 等角色，JOJO 是最難搞的，因為無敵的開啟時機、如何遞迴都是個難題，但只要想通了似乎也沒想像中那麼困難，相較於之前介紹的角色，JOJO 多了 stand_turns(剩餘無敵回合)、stand_used(是否開過無敵，因為無敵只能開一次)紀錄、描述無敵狀態。

```
// 替身狀態處理
if (stand_turns > 0)
{
    // 替身狀態可以走任何格子且不消耗生命值
    visited[nx][ny] = true;
    DFS_JOJO_Helper(Graph, visited, nx, ny, end, health,
                    stand_turns - 1, true, path_count);
    visited[nx][ny] = false;
}
else
{
    // 可以選擇在這步啟動替身
    if (!stand_used && health > 0)
    {
        visited[nx][ny] = true;
        DFS_JOJO_Helper(Graph, visited, nx, ny, end, health,
                        2, true, path_count);
        visited[nx][ny] = false;
    }

    // 正常移動處理
    if (Graph[nx][ny] == 0) // 不能走到樹木
        continue;

    // 計算新的生命值
    int new_health = health - 3; // 基本移動消耗
    if (Graph[nx][ny] == 2) // 踩到毒菇
    {
        new_health -= 10;
    }

    // 如果生命值足夠，繼續移動
    if (new_health > 0)
    {
        visited[nx][ny] = true;
        DFS_JOJO_Helper(Graph, visited, nx, ny, end, new_health,
                        0, stand_used, path_count);
        visited[nx][ny] = false;
    }
}
```

除了用 for 迴圈更新位置、邊界條件外，JOJO 需要在判斷是否為樹木前優先判斷是否為無敵狀態中定決定是否在這回合中開啟，這樣才能達成穿牆的效果，如果沒有的話在進行下一步。

輸入及輸出說明

```
string role;
int row, col;
pair<int, int> start, end;
while (in >> role)
{
    in >> row >> col;
    in >> start.first >> start.second;
    in >> end.first >> end.second;

    vector<vector<int>> Graph(row, vector<int>(col, 0));
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
        {
            in >> Graph[i][j];
        }
    }

    if (role == "luka")
    {
        out << DFS_LUKA(Graph, start, end) << endl;
    }
    else if (role == "fizz")
    {
        out << DFS_FIZZ(Graph, start, end) << endl;
    }
    else
    {
        out << DFS_JOJO(Graph, start, end) << endl;
    }
}
```

我透過 fstream library 的 ifstream、ofstream 讀入以及寫檔，然後再依據輸入格式，依序輸入角色、地圖大小(列、行)、起點、終點的 xy 座標，最後再輸入完整地圖，隨後程式會根據輸入的角色選用不同的 DFS 方案，並將結果輸出到 ofstream 所連結的檔案之中。

心得

在這學期中學到很多演算法，以前在高中在刷 Zerojudge 時，常常會看到解這道題目需要用到什麼程式解法，如果是一般的 bubble sort、binary search 等等的那還可以解決，但是只要一到需要更高級的演算法、排序法，這時候也就只能舉雙手投降，然後在這學期中學到了我以前所缺的，quick sort、merge sort、dynamic programming 等等技巧，在這學期的課程的幫助下以前的問題迎刃而解。

不過這次的作業的難度真的是太高了，前面的角色還好說，但 jojo 那無敵狀態真的是很搞，無敵三回合開在哪時候能提高生存率？整局不開的可能性為何？這些問題困擾我很久一段時間，但後來直接使用遞迴直接用 function call 代替 stack 運作，避免回朔時所造成的問題。